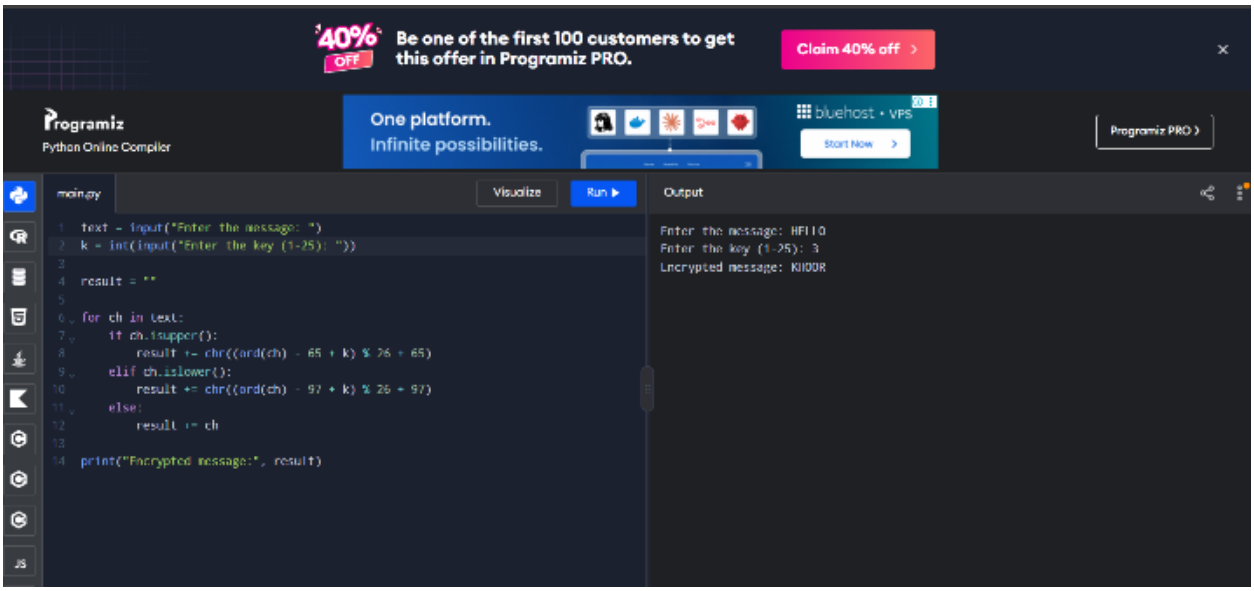
1. Write a C program for Caesar cipher involves replacing each letter of the alphabet with the letter standing k places further down the alphabet, for k in the range 1 through 25.

CODE:

```
text = input("Enter the message: ")
k = int(input("Enter the key (1-25): "))
result = ""
for ch in text:
    if ch.isupper():
        result += chr((ord(ch) - 65 + k) %26 + 65)
    elif ch.islower():
        result += chr((ord(ch) - 97 + k) %26 + 97)
    else:
        result += ch
print("Encrypted message:", result)
```

OUTPUT:



2. Write a C program for monoalphabetic substitution cipher maps a plaintext alphabet to a ciphertext alphabet, so that each letter of the plaintext alphabet maps to a single unique letter of the ciphertext alphabet.

CODE:

```
plain = "ABCDEFGHIJKLMNOPQRSTUVWXYZ"
cipher = "QWERTYUIOPASDFGHJKLZXCVBNM"
text = input("Enter the message: ").upper()
result = ""
for ch in text:
    if ch in plain:
        result += cipher[plain.index(ch)]
    else:
        result += ch
print("Encrypted message:", result)
```

OUTPUT:



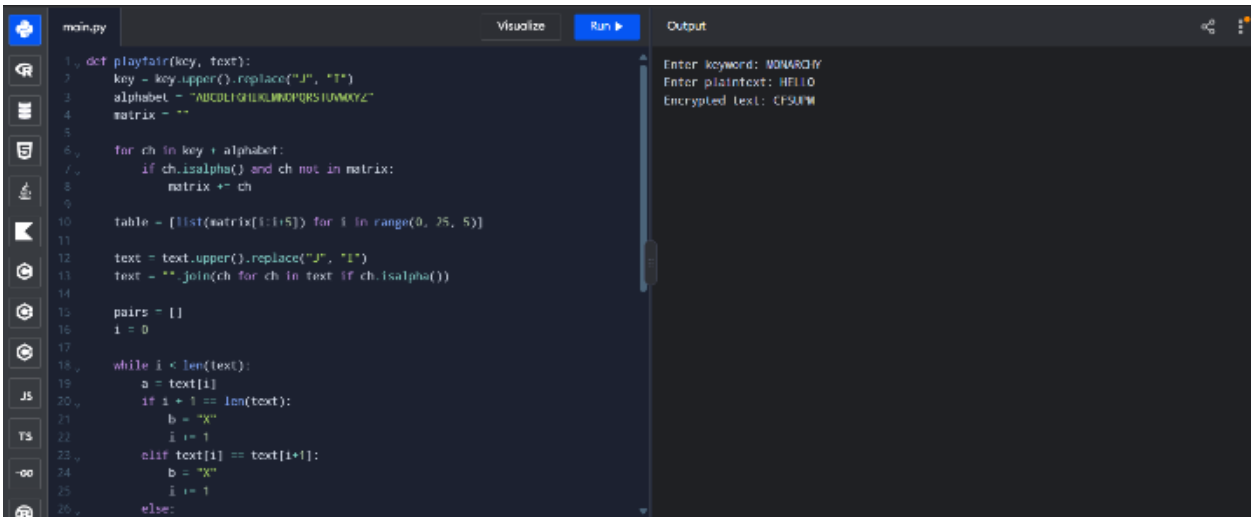
3. Write a C program for Playfair algorithm is based on the use of a 5 X 5 matrix of letters constructed using a keyword. Plaintext is encrypted two letters at a time using this matrix.

CODE:

```
def playfair(key, text):
    key = key.upper().replace("J", "I")
    alphabet = "ABCDEFGHIKLMNOPQRSTUVWXYZ"
    matrix = ""
    for ch in key + alphabet:
        if ch.isalpha() and ch not in matrix:
            matrix += ch
    table = [list(matrix[i:i+5]) for i in range(0, 25, 5)]
    text = text.upper().replace("J", "I")
    text = "".join(ch for ch in text if ch.isalpha())
    pairs = []
    i = 0
    while i < len(text):
        a = text[i]
        if i + 1 == len(text):
            b = "X"
            i += 1
        elif text[i] == text[i+1]:
            b = "X"
            i += 1
        else:
        b =text[i+1]
```

```
            i += 2
        pairs.append(a + b)
    result = ""
    for a, b in pairs:
        r1, c1 =next((r, c) for r in range(5) for c in range(5) if table[r][c] == a)
        r2, c2 =next((r, c) for r in range(5) for c in range(5) if table[r][c] == b)
        if r1 == r2:
            result += table[r1][(c1+1)%5] + table[r2][(c2+1)%5]
        elif c1 == c2:
            result += table[(r1+1)%5][c1] + table[(r2+1)%5][c2]
        else:
            result += table[r1][c2] + table[r2][c1]
    return result
key = input("Enter keyword: ")
text = input("Enter plaintext: ")
print("Encrypted text:", playfair(key, text))
```

OUTPUT:



4. Write a C program for polyalphabetic substitution cipher uses a separate monoalphabetic substitution cipher for each successive letter of plaintext, depending on a key.

CODE:

```
text = input("Enter plaintext: ").upper()

key = input("Enter key: ").upper()

result = ""

j = 0

for ch in text:

    if ch.isalpha():

        shift = ord(key[j %len(key)]) - 65

        result += chr((ord(ch) - 65 + shift) %26 + 65)

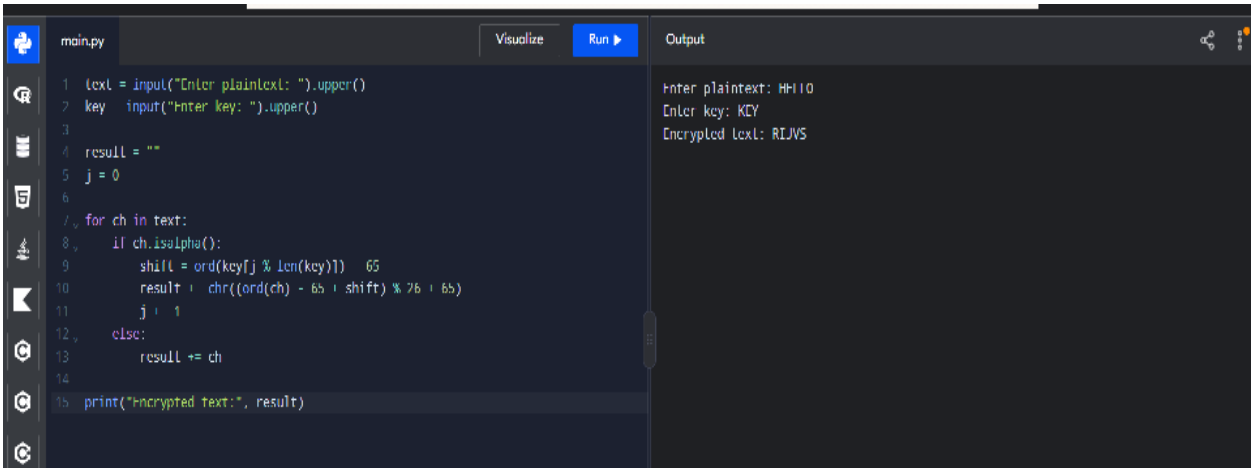
        j += 1

    else:

        result += ch

print("Encrypted text:", result)
```

OUTPUT:



5. Write a C program for generalization of the Caesar cipher, known as the affine Caesar cipher, has the following form: For each plaintext letter p, substitute the ciphertext letter C: C = E([a, b], p) =a(p + b) mod 26 A basic requirement of any

encryption algorithm is that it be one-to-one. That is, if p q, then E(k, p) E(k, q). Otherwise, decryption is impossible, because more than one plaintext character maps into the same ciphertext character. The affine Caesar cipher is not one-to-one for all values of a. For example, for a = 2 and b = 3, then E([a, b], 0) = E([a, b], 13) = 3. Are there any limitations on the value of b?

Determine which values of a are not allowed.

CODE:

```
text = input("Enter plaintext: ").upper()

a = int(input("Enter value of a: "))

b = int(input("Enter value of b: "))

if a < 1 or a > 25 or __import__('math').gcd(a, 26) != 1:

    print("Invalid value of a")

else:

    result = ""

    for ch in text:

        if ch.isalpha():

            p =ord(ch) - ord('A')

            c = (a * p + b) %26

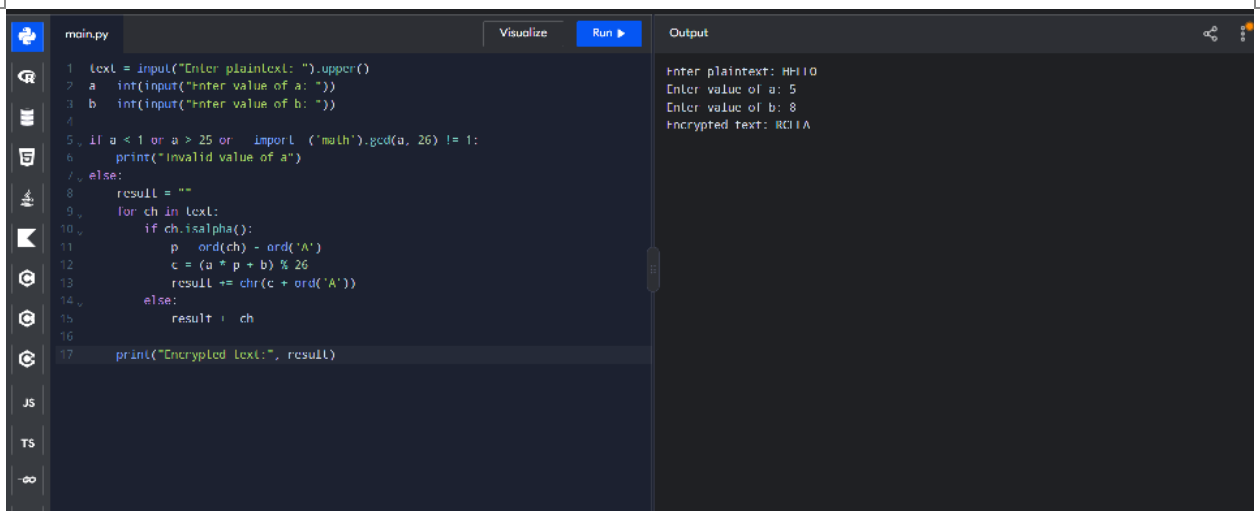
            result += chr(c + ord('A'))

        else:

            result += ch

    print("Encrypted text:", result)
```

OUTPUT:


main.py
Visualize
Run
Output
text = input("Enter plaintext: ").upper()
a = int(input("Enter value of a: "))
b = int(input("Enter value of b: "))
if a < 1 or a > 25 or import ('math').gcd(a, 26) != 1:
print("Invalid value of a")
else:
result = ""
for ch in text:
if ch.isalpha():
p ord(ch) - ord('A')
c = (a * p + b) % 26
result += chr(c + ord('A'))
else:
result + ch
print("Encrypted text:", result)
Enter plaintext: HELLO
Enter value of a: 5
Enter value of b: 8
Encrypted text: RCJJA